
A New Researcher's Companion to the Rabt System Paper

KnowledgeGraphEval 2026 • ArabicNLP 2026

Every technical concept, method, and model behind our system paper, explained from the ground up.

This document assumes you can read Python and have met neural networks once. It assumes nothing about named entity recognition, relation extraction, Arabic NLP, or shared tasks. Everything else is built up from scratch, with worked examples throughout.

Team Rabt • Salah Abdo, Faisal Muhammad Adam

Companion to `paper/latex/main.tex`

Contents

1	The problem	1
1.1	What is a knowledge graph?	1
1.2	Why two subtasks?	1
1.3	What makes each subtask hard here?	1
2	The foundation	2
2.1	Transformers and encoders in one page	2
2.2	Encoder-only vs decoder-only	2
2.3	Pretraining and fine-tuning	2
2.4	Why AraBERTv02?	3
3	Tokenization	3
3.1	Words are not tokens	3
3.2	The alignment problem	4
3.3	The 512-token limit and chunking	4
4	Subtask 1: NER	5
4.1	What NER is	5
4.2	BIO tagging: turning spans into per-token labels	5
4.3	The nesting problem	5
4.4	Our solution: 21 parallel BIO heads	6
4.5	What this architecture cannot do	7
5	The loss function	7
5.1	The imbalance problem	8
5.2	Cross-entropy (CE): the baseline	8
5.3	Dice loss: optimizing overlap instead of accuracy	8
5.4	Tversky loss: asymmetric overlap	8
5.5	Focal loss: focus on hard examples	9
5.6	The variance penalty: the clever part	9
5.7	One implementation detail worth knowing	10
6	Decoding	10
6.1	Invalid sequences and the repair rule	10
7	Ensembling for NER	11

7.1	Mean-probability	11
7.2	Span union (min_votes = 1)	11
7.3	Majority vote (min_votes = 2)	11
7.4	Resolving conflicts	12
7.5	What we learned, and why it mattered	12
8	Subtask 2: Relation Extraction	13
8.1	The task setup	13
8.2	The core problem: how does the model know which entities?	13
8.3	Reading the answer out	14
8.4	Two messy real-world details	14
8.5	The no_relation class and negative examples	14
8.6	Recovering entity types at test time	15
9	Constraint decoding	15
9.1	The idea	15
9.2	Building the table	16
9.3	Soft penalty, not a hard mask	16
9.4	Choosing the penalty	17
9.5	Result, and an interesting interaction	17
10	Cross-validation	17
10.1	k-fold cross-validation	17
10.2	Grouped splits and leakage	18
10.3	Out-of-fold prediction	18
10.4	Ensembling by logit averaging	18
11	Evaluation	19
11.1	Precision, recall, F1	19
11.2	What counts as correct for NER	19
11.3	Micro vs macro averaging	19
11.4	Why micro-F1 = precision = recall for our RE task	20
12	Domain shift	20
12.1	What it is	20
12.2	Measuring the tax	21
12.3	The shape of the failure	21
12.4	What would actually fix it	22
13	Experimental discipline	22

13.1 Never touch the test set	22
13.2 Model selection and early stopping	22
13.3 Seeds and reproducibility	23
13.4 Sanity gates before trusting a run	23
13.5 Counting your approximations	23
14 Glossary	24
15 Where to go next	25

1 The problem

1.1 What is a knowledge graph?

A **knowledge graph** stores facts as a graph: *things* are nodes, and *relationships between things* are edges. The fact “Birzeit University is located in Birzeit” becomes:

```
[Birzeit University] --located_in--> [Birzeit]
      (an ORG node)                (a GPE node)
```

Each node has a **type** (ORG = organization, GPE = geopolitical entity), and each edge has a **relation label** (Location, located_in). A knowledge graph built from a million news articles lets you answer questions no keyword search can: “which organizations are headquartered in cities that border Jordan?”

1.2 Why two subtasks?

Building a knowledge graph from raw text is traditionally split into two stages, and the shared task mirrors that split exactly:

```
Raw Arabic text
|
| Subtask 1: Named Entity Recognition (NER)
|   "which spans of text are entities, and of what type?"
v
Typed mentions: [Birzeit University]=ORG, [Birzeit]=GPE
|
| Subtask 2: Relation Extraction (RE)
|   "for a given pair of mentions, what relation holds?"
v
Typed edges: located_in(Birzeit University, Birzeit)
```

The two subtasks are scored separately, so we built two independent systems. In a real pipeline, RE would consume NER’s output and errors would compound; in the shared task, RE is given gold entity pairs, so the stages are decoupled.

1.3 What makes each subtask hard here?

AdaptNER is hard because of domain shift. We train on Wojood (mostly news text) but are tested on Konooz, drawn from ten domains never seen in training: agriculture, art, economics, finance, health, history, law, politics, science, sport. A model that memorizes “words that look like news entities” will fail on medical or agricultural text. Section 12 covers this properly, since it is the whole point of the subtask.

Relation extraction is hard because of label granularity. There are 40 relation types, and many are semantically adjacent: `Administration.president_of` VS `Administration.leader_of` VS `Affiliation.member_of` all describe a person connected to an organization. The model must make fine distinctions from a single sentence.

2 The foundation

2.1 Transformers and encoders in one page

A **transformer** processes a sequence of tokens and produces one vector per token. The key mechanism is **self-attention**: when computing the representation of a token, the model looks at every other token in the sequence and decides how much each one matters. This is why transformers handle context so well. The word “bank” gets a different vector in “river bank” than in “bank loan”, because attention pulls in the surrounding words.

The output for a sequence of T tokens is T vectors, each of size H (for BERT-base, $H = 768$). These are called **hidden states** or **contextual embeddings**. This is the crucial difference from older methods like word2vec, where each word had one fixed vector regardless of context.

2.2 Encoder-only vs decoder-only

- **Encoder-only** (BERT and relatives): every token sees every other token, in both directions. Great for *understanding* tasks where you need a rich representation of each input token. This is what we use.
- **Decoder-only** (GPT and relatives): each token sees only the tokens before it, because the model is trained to predict the next token. Great for *generation*.

Our tasks are labeling tasks (tag each token, classify each pair), not generation tasks. An encoder is the natural fit. The paper’s Discussion section reports that large generative models perform poorly on Arabic NER (0.29 to 0.74 F1 in published results), well below a fine-tuned BERT.

2.3 Pretraining and fine-tuning

Pretraining: a model is trained on enormous amounts of unlabeled text with a self-supervised objective. BERT’s objective is *masked language modeling*: hide 15% of tokens and predict them from context. This teaches the model syntax, semantics, and world knowledge without any human labeling.

Fine-tuning: take the pretrained model, attach a small task-specific layer (a **head**) on top, and continue training on your labeled data. The encoder’s weights update too, but gently (this is why our learning rates are tiny, 1e-5 and 2e-5, rather than the 1e-3 typical of training from scratch). We are nudging an already-competent model, not teaching it language from zero.

2.4 Why AraBERTv02?

We use `aubmindlab/bert-base-arabertv02` for both subtasks. Reasons:

1. **Arabic-specific.** Multilingual models spread their capacity across 100+ languages; a monolingual Arabic model spends all of it on Arabic.
2. **MSA-focused.** Our test set is Modern Standard Arabic (the organizers confirmed this). AraBERTv02 is trained mostly on MSA news and Wikipedia. MARBERT, an alternative, is trained on tweets, which suits dialectal Arabic but not our test set.
3. **Sensitivity matters.** Published Wojood-family results show roughly a 14.5 F1 spread across encoder choices. Encoder choice is one of the highest-leverage decisions in Arabic NER, so we followed the consensus rather than experimenting.
4. **Comparability.** The reference Wojood systems are AraBERT-based, so our numbers are comparable to published ones.

A note on Arabic. Arabic is morphologically rich: a single written word can bundle a preposition, an article, a stem, and a possessive suffix. Some Arabic NLP pipelines pre-segment words into morphemes (Farasa segmentation). We deliberately do *not*: our survey found aggressive preprocessing cost 7.2 F1 in comparable settings, and AraBERTv02’s own tokenizer already handles subword splitting. Sometimes the best preprocessing is none.

3 Tokenization

3.1 Words are not tokens

BERT does not operate on words. It uses a fixed vocabulary of roughly 30k **subword** units. Unknown or rare words are split into known pieces:

word	الجامعات (the universities)
subwords	ات## + الجامع

The `##` prefix means “this continues the previous token”. This solves the out-of-vocabulary problem: any word can be built from pieces, and related words share pieces.

3.2 The alignment problem

Our task labels **words**, but the model sees **subwords**. If a word splits into three subwords, which one gets the label?

Our convention (in `ner_data.py`): **label the first subword of each word; mark the rest with -100**, which is PyTorch’s “ignore this position” sentinel for loss functions. So:

word	الجامعات في فلسطين			
subword	الجامع	ات##	في	فلسطين
label	B-ORG	-100	0	B-GPE

The shaded position is ignored by the loss and at prediction time.

At prediction time we read the tag off the first subword and assign it to the whole word. This keeps a clean one-tag-per-word contract, which is what the submission format requires.

3.3 The 512-token limit and chunking

BERT has a maximum sequence length of 512 tokens, a hard architectural limit (positional embeddings only exist for 512 positions). Real documents are longer. Our approach (`chunking.py`):

- Split greedily at **word boundaries** with a budget of 510 subwords (leaving room for the [CLS] and [SEP] special tokens).
- **No overlap** between chunks.
- Reassemble predictions by (sentence, word offset) afterwards.

The tradeoff: a chunk boundary cuts the attention context, so an entity mentioned near a boundary loses part of its context. Overlapping windows would reduce this at the cost of ambiguity about which copy to trust. We chose the simpler option; sentences long enough to be split are rare.

4 Subtask 1: NER

4.1 What NER is

Named entity recognition finds spans of text that refer to entities and assigns each a type. Our task has 21 types (PERS, ORG, GPE, LOC, FAC, DATE, TIME, MONEY, CARDINAL, OCC, PRODUCT, LANGUAGE, LAW, QUANTITY, UNIT, NORP, EVENT, WEBSITE, and others).

A **span** is a contiguous range of words plus a type. We represent it as (start, end_exclusive, type), so words 3 and 4 tagged ORG is (3, 5, "ORG").

4.2 BIO tagging: turning spans into per-token labels

Neural sequence models naturally emit one label per token, not a list of spans. The standard bridge is **BIO tagging** (also called IOB2):

- B-X = Beginning of an entity of type X
- I-X = Inside (continuation) of an entity of type X
- O = Outside any entity

Worked example:

words	جامعة	بيرزيت	في	فلسطين
tags	B-ORG	I-ORG	O	B-GPE
one ORG span			one GPE span	
(0, 2, "ORG")			(3, 4, "GPE")	

The B vs I distinction matters for adjacency: two ORG entities right next to each other are B-ORG B-ORG, not B-ORG I-ORG, which would be one two-word entity.

4.3 The nesting problem

Nested entities are entities inside other entities. They are extremely common: 22.5% of Wojoood mentions are nested. In our example, “بيرزيت” (Birzeit, a city) is a GPE *inside* the ORG “جامعة بيرزيت” (Birzeit University):



The city is an entity *inside* the university, which is also an entity.

Now try to write that with one BIO tag per word:

words	جامعة	بیرزیت
tags	B-ORG	???

The shaded cell must be I-ORG (continuing the university) *and* B-GPE (starting the city). One column cannot hold both.

One tag column cannot hold two labels. This is the fundamental limitation that every nested-NER architecture exists to solve.

4.4 Our solution: 21 parallel BIO heads

Instead of one tag per word, we emit **21 tags per word, one per entity type**. Each type gets its own independent BIO column:

	PERS	ORG	GPE	DATE	...	(21 cols)
جامعة	0	B-ORG	0	0	...	
بیرزیت	0	I-ORG	B-GPE	0	...	

Read down the ORG column for the university; down the GPE column for the city. Both coexist because they occupy different columns.

Read down the ORG column: B-ORG, I-ORG gives the university. Read down the GPE column: 0, B-GPE gives the city. Both entities coexist because they live in different columns. Nesting is solved by construction.

Implementation (`ner_model.py`): this is not 21 separate neural networks. It is a single linear layer mapping the encoder's 768-dim hidden state to $21 \times 3 = 63$ outputs, then reshaped to `[batch, tokens, 21, 3]`. Each of the 21 groups of 3 is an independent softmax over `{O, B, I}`.

```
# conceptually:
logits = linear(hidden)           # [B, T, 63]
```

```
logits = logits.view(B, T, 21, 3)    # [B, T, 21, 3]
# logits[b, t, k, :] = the O/B/I scores for type k at token t
```

This is elegant for three reasons:

1. **It matches the submission format exactly.** The official format is 21 tag columns per token. Our model emits 21 tag columns per token. No conversion code, no conversion bugs.
2. **It is cheap.** One shared encoder pass, one small linear layer. A competing approach (MRC-style querying, “where are the ORGs in this sentence?” asked 21 times) would cost 21 encoder passes.
3. **The heads share an encoder**, so learning to recognize organizations helps the model represent locations too. This is multi-task learning.

4.5 What this architecture cannot do

It cannot represent **two entities of the same type that overlap**, for example an ORG inside another ORG. Both would need the same column, and a column holds one BIO chain.

This is worth understanding as a general lesson: *every representation choice forecloses something*. We accepted this because:

- The official 21-column submission format has the same limitation, so no architecture could express it anyway;
- the organizers confirmed Konooz contains no same-type nesting;
- in Wojood only about 576 pairs are affected.

When a collision does occur, we keep the outer span, drop the inner, and count the drop. **Counting your known failures is not optional.** An uncounted approximation is a silent bug.

5 The loss function

A **loss function** measures how wrong the model is; training minimizes it. We use a weighted combination of four losses, following the UM6P&UL system from Wojood-NER 2023:

$$L_{\text{head}} = 0.4 \cdot \text{CE} + 0.2 \cdot \text{Dice} + 0.2 \cdot \text{Tversky} + 0.2 \cdot \text{Focal}$$

Why four? Because they fail differently, and the thing they are fighting is **class imbalance**.

5.1 The imbalance problem

In any NER dataset, 0 massively outnumbers B and I. In our per-type formulation it is far worse: for the WEBSITE column, almost every token in the corpus is 0. A model that predicts 0 everywhere in that column scores 99.9% accuracy and is completely useless.

5.2 Cross-entropy (CE): the baseline

Standard classification loss. For the correct class with predicted probability p , the loss is $-\log p$. Confident and correct gives near zero; confident and wrong gives a large penalty.

Weakness: it optimizes average per-token correctness. With 99.9% 0 tokens, predicting all-0 is an excellent CE solution.

5.3 Dice loss: optimizing overlap instead of accuracy

The **Dice coefficient** measures overlap between predicted and true sets:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

This is exactly the F1 score in set form. Since we are *evaluated* on F1, optimizing an F1-shaped objective directly is appealing. $\text{Loss} = 1 - \text{Dice}$.

Crucially, Dice is **normalized by set size**, so a rare type contributes as much as a frequent one. The all-0 solution scores terribly on Dice for the entity classes, because the intersection is empty.

To keep it differentiable we use predicted *probabilities* rather than hard decisions (a “soft” Dice).

5.4 Tversky loss: asymmetric overlap

The **Tversky index** generalizes Dice by weighting false positives and false negatives differently:

$$\text{Tversky} = \frac{TP}{TP + \alpha \cdot FP + (1 - \alpha) \cdot FN}$$

- $\alpha > 0.5$: penalize false positives more, pushing toward **precision**.
- $\alpha < 0.5$: penalize false negatives more, pushing toward **recall**.
- $\alpha = 0.5$: symmetric, and mathematically **equivalent to Dice**.

An honest observation. We use $\alpha = 0.5$, which makes the Tversky term equivalent to a linear Dice. Our Dice term uses the squared-denominator (V-Net) variant, so the two terms are not literally identical, but they are measuring the same thing and are highly correlated. In effect our loss is closer to “0.4 CE + 0.4 Dice-ish + 0.2 Focal” than to four genuinely independent objectives. We inherited the recipe from a system that reported +1.18 F1 with it, and did not ablate it.

This is the most interesting unexplored knob in our system. Our single biggest weakness is recall collapse under domain shift (Section 12). Tversky’s α is *precisely* the dial for trading precision against recall, and we left it at the neutral setting. Trying $\alpha = 0.3$ is an obvious experiment, and a good first contribution for a new team member.

5.5 Focal loss: focus on hard examples

Introduced for object detection, **focal loss** modifies CE by down-weighting examples the model already gets right:

$$\text{Focal} = (1 - p_t)^\gamma \cdot (-\log p_t)$$

where p_t is the predicted probability of the correct class and $\gamma = 2$.

Read the multiplier: if the model already assigns $p_t = 0.99$ to the right answer, then $(1 - 0.99)^2 = 0.0001$, so that example contributes almost nothing. If $p_t = 0.1$, the multiplier is 0.81, nearly full weight. Training effort concentrates on the hard cases instead of being swamped by millions of easy 0 tokens.

5.6 The variance penalty: the clever part

Each of the 21 heads produces its own loss. The obvious way to combine them is to average. We do something more:

$$L_{\text{total}} = \text{mean}(L_{\text{heads}}) + 5.0 \cdot \text{Var}(L_{\text{heads}})$$

Why? Averaging lets the model trade a large improvement on frequent types (PERS, ORG, GPE) against neglect of rare ones (WEBSITE, LAW, UNIT). The average would look fine while several heads never learned anything.

The variance term punishes *disagreement between heads*. If 20 heads have low loss and one has high loss, the variance is large and the penalty bites. The only way to reduce it is to bring the straggler up. It is a fairness constraint across the 21 sub-problems, and with weight 5.0, a strong one.

This is a genuinely transferable idea: **when your loss is a sum over sub-problems you care about individually, penalize the variance, not just the mean.**

5.7 One implementation detail worth knowing

The loss is computed in **fp32 even though training uses fp16** (`ner_model.py`). Mixed-precision training uses 16-bit floats for speed, but Dice and Tversky sum over the entire batch, and long sums of small 16-bit numbers lose precision catastrophically. CE and focal are per-token means and tolerate fp16 fine; the set-level losses do not. Casting just the loss computation to fp32 costs nothing measurable and avoids silent numerical corruption.

6 Decoding

Decoding converts model output back into entities. For each of the 21 columns independently (`spans.py`):

```
tags: 0 B-ORG I-ORG 0 B-ORG
      -> spans (1,3,"ORG") and (4,5,"ORG")
```

Straightforward, except when the model emits something invalid.

6.1 Invalid sequences and the repair rule

The 21 heads predict each token independently. Nothing forces the output to be a valid BIO sequence, so the model can produce an **orphan I-**:

```
tags: 0 I-ORG I-ORG 0
      ^^^^^ "continue an ORG", but no ORG was started
```

Options:

1. Drop it (lose a probably-real entity),

2. treat the orphan as a B- and start a span there,
3. use constrained decoding (Viterbi/CRF) to force validity.

We chose option 2: **an orphan I-x starts a new span**, and every repair is counted. On the blind test this fired 201 times out of 5,780 spans, about 3.5%. Reporting that number is what makes the choice defensible: it is a measured approximation, not a hidden hack.

Why not a CRF? A conditional random field layer would learn transition scores between tags and use Viterbi decoding to guarantee valid sequences. It is the textbook answer. It also costs 21 CRFs (one per column), slows training, and buys little when only 3.5% of spans need repair. Worth trying, but not obviously worth the complexity.

7 Ensembling for NER

An **ensemble** combines several models to beat any individual one. The principle: if models make *independent* errors, combining them cancels noise while preserving signal.

We trained three models identical in every way except the **random seed** (13, 42, 77). The seed controls weight initialization, dropout, and data order. Same data, same hyperparameters, three different local optima.

We tried three combination strategies:

7.1 Mean-probability

Average the softmax probability distributions across seeds, then argmax. Combines at the level of *model confidence*, letting a very confident model outvote two uncertain ones.

7.2 Span union (**min_votes = 1**)

Decode each model to spans independently, keep every span **any** model predicted.

- More predictions, so **higher recall, lower precision**.
- A *recall lever*.

7.3 Majority vote (**min_votes = 2**)

Keep only spans that **at least 2 of 3** models predicted.

- Fewer predictions, and each is corroborated: **higher precision**.
- A *precision lever*.

Worked example, where the three seeds disagree:

```
seed 13: (0,2,ORG) (5,6,GPE)
seed 42: (0,2,ORG) (5,6,GPE) (9,11,PERS)
seed 77: (0,2,ORG) (9,11,PERS)

vote counts: (0,2,ORG)=3 (5,6,GPE)=2 (9,11,PERS)=2

union (>=1): all three spans
vote2 (>=2): all three spans (all reached 2)
vote3 (>=3): only (0,2,ORG)
```

7.4 Resolving conflicts

Union can produce two overlapping spans **of the same type**, which the output format cannot encode (Section 4). We resolve greedily, sorting candidates by (votes desc, length desc, start asc) and taking non-conflicting ones. On the blind test this dropped 698 candidates from the union variant, and zero from the vote variant. Deterministic given input order, so runs are reproducible.

7.5 What we learned, and why it mattered

In-domain results (Wojood test):

Variant	P	R	F1
Single seed	0.9183	0.9353	0.9268
Mean-prob	0.9211	0.9379	0.9295
Union	0.8981	0.9440	0.9205
Majority vote	0.9281	0.9377	0.9329

We *wanted* the union to be the answer, because our blind-set weakness is recall (Section 12). But we ran a diagnostic first: **the union predicts only 8% more spans on the blind set than the best single seed** (6,479 vs 5,982).

That single number changed the decision. If the seeds nearly agree on blind data, then the entities we are missing are missed by *all* seeds: the failure is **systematic**, not random. Ensembling cancels random error; it cannot recover something no member ever finds. The recall lever had nothing to lever.

So we submitted the vote, and the blind results confirmed the reasoning: +1.9 precision, +0.3 recall, +0.98 F1 overall.

The transferable lesson: before deploying a fix, measure whether the problem it addresses is actually the problem you have.

8 Subtask 2: Relation Extraction

8.1 The task setup

Given a sentence and **two marked mentions** (a subject and an object), output which of 40 relations holds. Note what is *not* asked: we do not find the entities (they are given), and we do not decide *whether* a relation exists.

This last point is important. The test set contains only positive pairs, so the model must always choose some relation. This is called **forced choice**, and it has a consequence explored in Section 11: precision, recall, and F1 all become the same number.

8.2 The core problem: how does the model know which entities?

The sentence is just a string of tokens. If we feed it to BERT unchanged, nothing indicates which two mentions we are asking about. There are several solutions; the standard modern one is **entity markers**: insert special text around each mention.

We use **typed punctuation markers** (Zhou and Chen, 2022):

original	عملت في جامعة بيرزيت في مدينة بيرزيت
marked	# بيرزيت ^ gpe ^ # في مدينة @ جامعة بيرزيت * org * @ عملت في

Subject markers wrap the university; **object markers** wrap the city. Each carries its entity type.

Three design details, each with a reason:

1. **The markers carry the entity types** (org, gpe), not just positions. Type information is a strong signal: a PERS-ORG pair cannot be `has_border_with`.
2. **They are punctuation, not new vocabulary tokens.** An alternative is adding real tokens like `[E1]` to the tokenizer, but those start with random embeddings and need enough data to learn. At 17k examples, reusing characters BERT already understands is more sample-efficient.

3. **They are inline text**, so self-attention handles them natively. No architectural change is needed.

8.3 Reading the answer out

After encoding the marked sentence, we take the hidden states at the two **start markers** (@ and #), concatenate them into one $2H$ vector, and pass that through a small feed-forward classifier:

```
Linear(2H -> H) -> ReLU -> Dropout -> Linear(H -> 41)
```

Why the start markers rather than the mention tokens? Because after self-attention, the marker position has aggregated information about the mention it wraps *and* its context. It acts as a summary slot. This is the standard finding from the marker literature and it saves us from having to pool over variable-length mentions.

8.4 Two messy real-world details

Ambiguous occurrences (5.9% of instances). If the mention string “بيرزيت” appears three times in the sentence, which occurrence do we mark? We choose the disjoint subject-object pair with the **smallest distance between them**, breaking ties leftmost. The intuition is that a stated relation usually involves nearby mentions.

Containment (about 20% of instances). Frequently the object is *inside* the subject, as in `located_in`(Birzeit University, Birzeit). Naive marking would corrupt the text. We **nest** the markers instead:

@ * org * جامعة # ^ gpe ^ بيرزيت # @

Nested markers: the object sits inside the subject, and both marker pairs stay well-formed.

Both mentions stay readable, and both marker pairs are well-formed.

8.5 The `no_relation` class and negative examples

Training data includes **negative pairs** (mention pairs with no relation), labeled `no_relation`. That gives 41 classes: 40 real relations plus `no_relation`.

But the test set is forced-choice positive, so emitting `no_relation` is always wrong. Two options:

1. Drop negatives from training and train a 40-way classifier.
2. Train on all 41 classes, then forbid `no_relation` at prediction time.

We chose (2), setting the `no_relation` logit to $-\infty$ before the `argmax` so it can never win. Why keep negatives we will never predict? Because they are **informative**: they teach the model what an unrelated pair looks like, sharpening the boundary for real relations. Published results indicate downsampling negatives costs about 10 F1.

This is worth internalizing: **training-time and inference-time label spaces do not have to match**. Train on everything informative; constrain the output to what is valid.

8.6 Recovering entity types at test time

Our markers need entity types, but the blind test provides mentions without types. We recover them with a three-stage fallback chain (`re_typing.py`):

1. **Sentence join.** Each RE record carries a sentence id. Join it against Wojood's gold sentences, then match the mention string to the gold entities in *that* sentence, taking the majority type on ties.
2. **Lexicon fallback.** If the join fails, look the mention string up in a corpus-wide string-to-type dictionary built from all Wojood splits.
3. **UNK.** If both fail, mark the type unknown. That side gets an untyped marker and its constraints are switched off (Section 9).

Results on the blind test: the join located a source sentence for 4,360 of 4,386 records (99.4%); after the lexicon, 4,290 subjects (97.8%) and 4,295 objects (97.9%) had types; 96 subject and 91 object sides remained unknown.

Is using gold Wojood entities here cheating? No, and the reasoning is worth following. The banned thing is training on Konooz, or using Konooz labels. Wojood is our *training* corpus, given to us with gold annotations, and RE is a separate subtask from NER. We are not predicting entities, we are looking up type metadata for entity pairs the task itself handed us. It is a legitimate use of provided resources, and we document it explicitly so readers can judge for themselves. When a protocol question is genuinely arguable, the right move is to state exactly what you did rather than let a reader assume.

9 Constraint decoding

9.1 The idea

Relations have **type signatures**. `GPE.has_population` connects a place to a number; it cannot connect two people. `Family.has_parent` connects two people; it cannot connect

an organization to a date. In ontology terms, each relation has a **domain** (allowed subject types) and a **range** (allowed object types).

A neural classifier does not know this. It might assign high probability to `has_population(PERS, PERS)` simply because the sentence looks vaguely demographic. Since we already recovered entity types, we can rule such predictions out.

9.2 Building the table

Our admissibility table (`re_constraints.py`) is the **union** of two sources:

1. **The specification:** the task’s published domain/range for each relation, hand-transcribed. For each relation, the allowed (subject type, object type) pairs are the cartesian product of its domain and range.
2. **Observation:** every (subject type, object type) pair that actually appears with that relation in training data.

The union matters, and the direction of the union matters more: **observation extends the specification, never narrows it**. Real annotations sometimes violate the published spec. If we trusted only the spec we would penalize predictions that the gold data itself contains. When your prior and your data disagree, and the data is what you are scored against, trust the data.

9.3 Soft penalty, not a hard mask

Two ways to apply constraints:

- **Hard mask:** set inadmissible logits to $-\infty$. They become impossible.
- **Soft penalty:** subtract a constant from inadmissible logits. They become unlikely, but a sufficiently confident model can still override.

We use a soft penalty of 3.0 logits. **Why not the hard mask?** Because our table is imperfect. Gold annotations violate the spec, our recovered types can be wrong, and rare-but-valid combinations may be missing from training. A hard mask makes every table error unrecoverable. A soft penalty says “this looks wrong, be sure”, which is the appropriate epistemic stance for a heuristic.

Additionally, if either entity type is UNK, we apply **no penalty at all**. Unknown types should not produce confident constraints.

9.4 Choosing the penalty

We swept the penalty over {0, 1, 2, 3, 4} and picked 3.0. Note we tuned it on development data, never on the test set, and re-validated it later on a much larger pooled sample (Section 10).

9.5 Result, and an interesting interaction

Setting	micro-F1	macro-F1
Single model, no constraints	0.9344	0.649
Single model + penalty 3.0	0.9470	0.705
5-fold ensemble, no constraints	0.9592	0.638
5-fold ensemble + penalty 3.0	0.95995	0.648

Constraints are worth **+1.26 F1 to a single model but only +0.08 to the ensemble**. Why the collapse?

Because the two techniques fix the *same errors*. A type-violating prediction is usually one model's fluke; average five models and the fluke is already outvoted. By the time constraints are applied, the ensemble has cleaned up most of what they would have caught.

But look at macro-F1: constraints add +1.0 there even on the ensemble. Macro-F1 weights every relation equally, so it is dominated by **rare** relations, where the ensemble has little training signal and the type system still helps.

Two lessons. First, techniques that address the same failure mode do not stack, so always measure your improvements *in combination*, not individually. Second, an intervention can look worthless on your headline metric while doing real work in the tail. Report both.

10 Cross-validation

10.1 k-fold cross-validation

Rather than one train/dev split, **k-fold** splits data into k parts, trains k models, and gives each a different held-out part:

```
fold 1: [TEST][ ][ ][ ][ ]
fold 2: [ ][TEST][ ][ ][ ]
```

```
...
fold 5: [ ] [ ] [ ] [ ] [TEST]
```

Every record gets predicted exactly once by a model that never saw it. Benefits: a more reliable estimate (5 measurements, not 1), and five trained models you can ensemble for free.

10.2 Grouped splits and leakage

Data leakage is when information from your evaluation set reaches training, inflating your score and misleading you about real performance.

Our RE data has a subtle leakage risk: one sentence generates **multiple relation records** (2.75 on average). Split randomly by record, and the same sentence appears in both train and dev with different mention pairs. The model has effectively seen the dev sentence.

The fix is **grouped k-fold**: group records by `sentence_id` and assign whole groups to folds (`splits.py`). A sentence lands entirely in one fold. We balance fold sizes greedily since groups vary in size.

Generalize this. Whenever your rows are not independent, split by the unit of independence, not the row. Multiple readings of one patient, multiple frames of one video, multiple records of one sentence: same trap, same fix.

10.3 Out-of-fold prediction

Out-of-fold (OOF) predictions are what you get by collecting each fold's held-out predictions and pooling them. The result is a prediction for *every* training record, each made by a model that did not train on it.

We used this to re-validate the constraint penalty. Originally we tuned it on a single 2,174-record dev split. With OOF, we re-ran the sweep on all **17,381** records: eight times the evidence, no extra training. Penalty 3.0 held up.

The OOF estimate (0.95995) also predicted our blind test score (0.9571) to within 0.3 points. That is a meaningful validation of the whole protocol: a leakage-free estimate that transfers to unseen data is doing its job.

10.4 Ensembling by logit averaging

For test predictions we average the five folds' **raw logits** (the pre-softmax scores), then apply the constraint penalty and the `no_relation` mask.

Averaging logits rather than probabilities keeps confidence information that softmax would compress. Order matters too: average first, *then* constrain, so the penalty applies once to a consensus rather than five times to individuals.

We also **export the averaged logits to a file**. Re-deciding the penalty later becomes a two-second CPU operation instead of a 68-minute GPU rerun. Saving the expensive intermediate is one of the highest-value habits in experimental ML.

11 Evaluation

11.1 Precision, recall, F1

For any set of predictions:

- **True positive (TP)**: predicted, and correct.
- **False positive (FP)**: predicted, but wrong.
- **False negative (FN)**: should have been predicted, but was missed.

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN} \quad F_1 = \frac{2PR}{P + R}$$

Precision answers “when I make a claim, how often am I right?” **Recall** answers “of everything I should have found, how much did I find?” **F1** is their harmonic mean, which (unlike an arithmetic mean) stays low if either is low. You cannot game F1 by maximizing one and ignoring the other.

11.2 What counts as correct for NER

We use **exact match**: a predicted span is a TP only if *both* its boundaries and its type are exactly right. Predicting “جامعة بيرزيت” as GPE instead of ORG is wrong. Predicting only “بيرزيت” when the gold span is “جامعة بيرزيت” is wrong (and counts twice: one FP and one FN).

This is strict. Partial-credit schemes exist, but exact match is the shared task standard and avoids arguments about partial credit.

11.3 Micro vs macro averaging

With 21 entity types (or 40 relations), you have many per-class scores. Two ways to summarize:

- **Micro:** pool all TP/FP/FN across classes, then compute one F1. Every *instance* counts equally, so frequent classes dominate.
- **Macro:** compute F1 per class, then average the F1s. Every *class* counts equally, so rare classes matter as much as common ones.

They answer different questions. Micro asks “how well does the system do on typical input?” Macro asks “does the system work across the board, or only on the easy majority?” A big micro-macro gap (ours: 0.7483 vs 0.6316) means rare types are being handled much worse than common ones.

Always report both. Reporting only micro hides neglected classes; reporting only macro lets one well-handled rare class flatter you.

11.4 Why micro-F1 = precision = recall for our RE task

This surprises people, so here is the derivation. In forced choice, the system outputs exactly one label per test instance, and every instance has exactly one gold label. So:

- number of predictions = number of gold labels = N
- every prediction is either right (TP) or wrong (FP)
- every wrong prediction is *also* a missed gold label (FN)

Therefore $FP = FN$, and:

$$P = \frac{TP}{TP + FP} = \frac{TP}{N} \quad R = \frac{TP}{TP + FN} = \frac{TP}{N}$$

Since $P = R$, their harmonic mean equals the same value. All three are simply **accuracy**. This is why our RE results table shows “=micro” for precision and recall: it is not laziness, it is an identity.

It also explains why NER is different: NER systems choose *how many* spans to predict, so predictions and gold counts differ, $FP \neq FN$, and P and R diverge meaningfully.

12 Domain shift

12.1 What it is

Machine learning assumes train and test data are drawn from the same distribution. **Domain shift** is when they are not. Our training text is Wojoood news; our test text is

Konooz agriculture, health, law, and seven other domains.

Different domains differ in vocabulary (crop names, medical terms, legal citations), in which entity types are common, in sentence style, and in which surface forms signal an entity.

12.2 Measuring the tax

Setting	micro-F1	P	R
Wojood test (in-domain)	0.9240	0.9169	0.9313
Konooz blind (out-of-domain)	0.7385	0.7874	0.6953

An **18.6-point drop**. This is not a bug in our system; it is the phenomenon the subtask was designed to measure. Konooz’s own corpus paper reports Arabic NER models losing 12 to 20 points across domains, so we land at the expected end of the published band.

12.3 The shape of the failure

Look at precision and recall separately, because the *shape* of the drop is more informative than its size:

in-domain:	P 0.9169	R 0.9313	(recall HIGHER than precision)
out-of-domain:	P 0.7874	R 0.6953	(recall 9.2 points LOWER)

The relationship flips. In domain, the model finds slightly more than it should; out of domain, it finds far too little.

Interpretation: when the model does make a prediction on new-domain text, it is still usually right (precision holds up reasonably). The failure is that it stays silent. Confronted with an unfamiliar agricultural term, it does not mislabel it, it simply fails to recognize it as an entity at all.

This diagnosis directly determined our strategy. Recall was the bottleneck, so we needed a recall lever. That is what motivated the union ensemble, and Section 7 tells the rest of the story: the diagnostic showed seeds agreed on the blind set, so seed ensembling could not supply the missing recall, and we chose the precision-preserving vote instead.

12.4 What would actually fix it

Since the deficit is systematic, the fix must inject new *knowledge* of the target domains, not new combinations of the same model. Designed but not built within the deadline:

- **DAPT (domain-adaptive pretraining)**: continue masked-language-model pretraining on the unlabeled test-domain text. No labels needed, so it is permitted. The model learns agricultural and medical vocabulary before ever being asked to tag it. Projected +2 to +5.
- **Self-training**: predict on unlabeled target text, keep high-confidence predictions as pseudo-labels, retrain. Projected +1 to +2. Risky (it amplifies its own errors), hence “guarded”.
- **LLM data augmentation**: generate synthetic in-domain training sentences. Explicitly recommended by the organizers.

All three are legitimate under the rules, which ban Konooz *labels*, not the released unlabeled text. Using unlabeled test inputs at training time is called **transductive learning** and the organizers sanctioned it. This is the clearest direction for future work.

13 Experimental discipline

The methods above are ordinary. What makes results trustworthy is protocol, so this section is arguably the most transferable part of the document.

13.1 Never touch the test set

Konooz never entered training, validation, or model selection. Every decision was made on Wojood-internal splits.

Why so strict, when peeking would be undetectable? Because each peek launders test information into your choices. Tune a hyperparameter on the test set and your reported number is no longer an estimate of generalization; it is a measure of how hard you searched. The number stops being *about* anything.

13.2 Model selection and early stopping

Early stopping trains until a validation metric stops improving, then keeps the best checkpoint. It prevents **overfitting** (memorizing training data instead of learning patterns).

Note the subtlety in our ensemble runs. We trained on train+validation combined, which means there was no held-out validation data left, which means early stopping was impossible *by construction*. So we fixed the schedule at 29 epochs, the epoch where the earlier train-only model had peaked, whose curve was flat between epochs 26 and 33.

The reasoning chain matters: we did not silently drop early stopping. We noted that our design forfeited it, borrowed the schedule from a comparable run, and verified the target region was flat so small errors would not hurt. **When you give up a safeguard, say so and explain what replaces it.**

13.3 Seeds and reproducibility

Neural training is stochastic. Fixing the random seed makes a run reproducible. Our seeds (13, 42, 77) are recorded for every experiment, alongside git commit hashes and data checksums, in `experiments/ledger.tsv`.

The three-seed spread is itself informative: our seeds scored 0.9248, 0.9253, 0.9268. That roughly 0.2-point spread is the noise floor. Any “improvement” smaller than that is indistinguishable from luck, which is why single-run comparisons of small deltas are untrustworthy.

13.4 Sanity gates before trusting a run

Our Kaggle jobs write a completion sentinel with pass/fail checks, and we refuse to use output unless it passes: every seed above a minimum F1, the union a superset of individual seeds, a minimum number of predicted spans, and a full submission-format rehearsal. Catching a broken run before it becomes a result is much cheaper than discovering it after it becomes a table in your paper.

13.5 Counting your approximations

Notice the recurring pattern:

- IOB repairs: **201 counted**
- Same-type overlaps dropped: **698 counted**
- Untyped entity sides: **96 subjects, 91 objects counted**
- Truncated markers: **0 counted**

Every place we approximate, we count and report. This is what separates a documented limitation from a hidden bug. A reader can judge whether an approximation invalidates our conclusions only if we tell them how often it fired.

14 Glossary

Term	Meaning
Ablation	Removing a component to measure its contribution
Argmax	The index of the largest value; turns scores into a decision
BIO / IOB2	Tagging scheme: Beginning, Inside, Outside
Checkpoint	Saved model weights at a point in training
Contextual embedding	Token vector that depends on surrounding words
CRF	Layer enforcing valid label sequences via transition scores
DAPT	Domain-adaptive pretraining: more MLM on target-domain text
Dev / validation set	Held-out data used for tuning, not final scoring
Domain shift	Train and test data come from different distributions
Dropout	Randomly zeroing activations during training, to regularize
Early stopping	Halt training when validation stops improving
Encoder	Model mapping tokens to contextual vectors
Ensemble	Combining several models' predictions
Epoch	One full pass over the training data
F1	Harmonic mean of precision and recall
Fine-tuning	Adapting a pretrained model to a task with labeled data
fp16 / fp32	16- and 32-bit floating point
Forced choice	Every instance must receive a label; no abstention
Head	Small task-specific layer on top of an encoder
Hidden state	An encoder's output vector for one token
Leakage	Evaluation information contaminating training
Logit	Raw pre-softmax score
Macro-average	Average of per-class scores; classes weighted equally
Micro-average	Pool all instances, then score; instances weighted equally
MLM	Masked language modeling, BERT's pretraining objective
MSA	Modern Standard Arabic, the formal written register
Nested entity	An entity contained inside another entity

Term	Meaning
OOF	Out-of-fold: predictions from a model that never saw the record
Overfitting	Memorizing training data instead of generalizing
Pseudo-label	Model prediction used as a training label
Seed	Random-number initializer; fixing it makes runs reproducible
Self-attention	Mechanism letting each token attend to all others
Softmax	Turns logits into a probability distribution
Span	Contiguous token range plus a type
Subword	Vocabulary unit smaller than a word
Transductive learning	Using unlabeled test inputs during training
Zero-shot	Applying a model to a setting it saw no training data for

15 Where to go next

To understand the evaluation more deeply, read `concepts.md` in this folder: worked scoring examples and the exact official metric definitions.

To understand the task theoretically, read `knowledgegrapeval_2026_theoretical_foundations.md`: ontology design, domain adaptation theory, and evaluation theory, independent of our implementation.

To read the code, this is the order that will make sense:

1. `columns.py` (the 21 types, and their fixed ordering)
2. `spans.py` (BIO decoding and the ensemble vote; small and self-contained)
3. `ner_model.py` (the architecture and the unified loss)
4. `re_markers.py` (marker insertion, including the nesting case)
5. `re_constraints.py` (the type table and penalty logic)
6. `kaggle/*/kernel/*.py` (how a full campaign is orchestrated)

The test suite in `tests/` is also good reading: tests are executable documentation of what each component promises.

To contribute an experiment, the open questions we consider most promising:

1. **Tversky $\alpha < 0.5$** (Section 5). The loss has a precision/recall dial sitting at neutral while recall is our known weakness. Cheap to try, directly targets the deficit.
2. **DAPT on the unlabeled test text** (Section 12). The largest projected gain (+2 to +5), organizer-sanctioned, and it attacks the systematic recall failure at its root.
3. **A CRF layer or constrained decoding** (Section 6) to eliminate the 201 repairs rather than patching them.
4. **Reconciling our scorer with the official one** (Section 13). We know our local numbers are self-consistent but have not verified they match the official implementation exactly.

Anything you try, add a row to `experiments/ledger.tsv`: run id, git hash, config, seed, metric, score. Future you will need it, and so will the paper.